CSC 340-02

Programming Methodology
Week 6

- Review Weeks 1-5

Instructor: Joseph Hui

Email: jhui@sfsu.edu

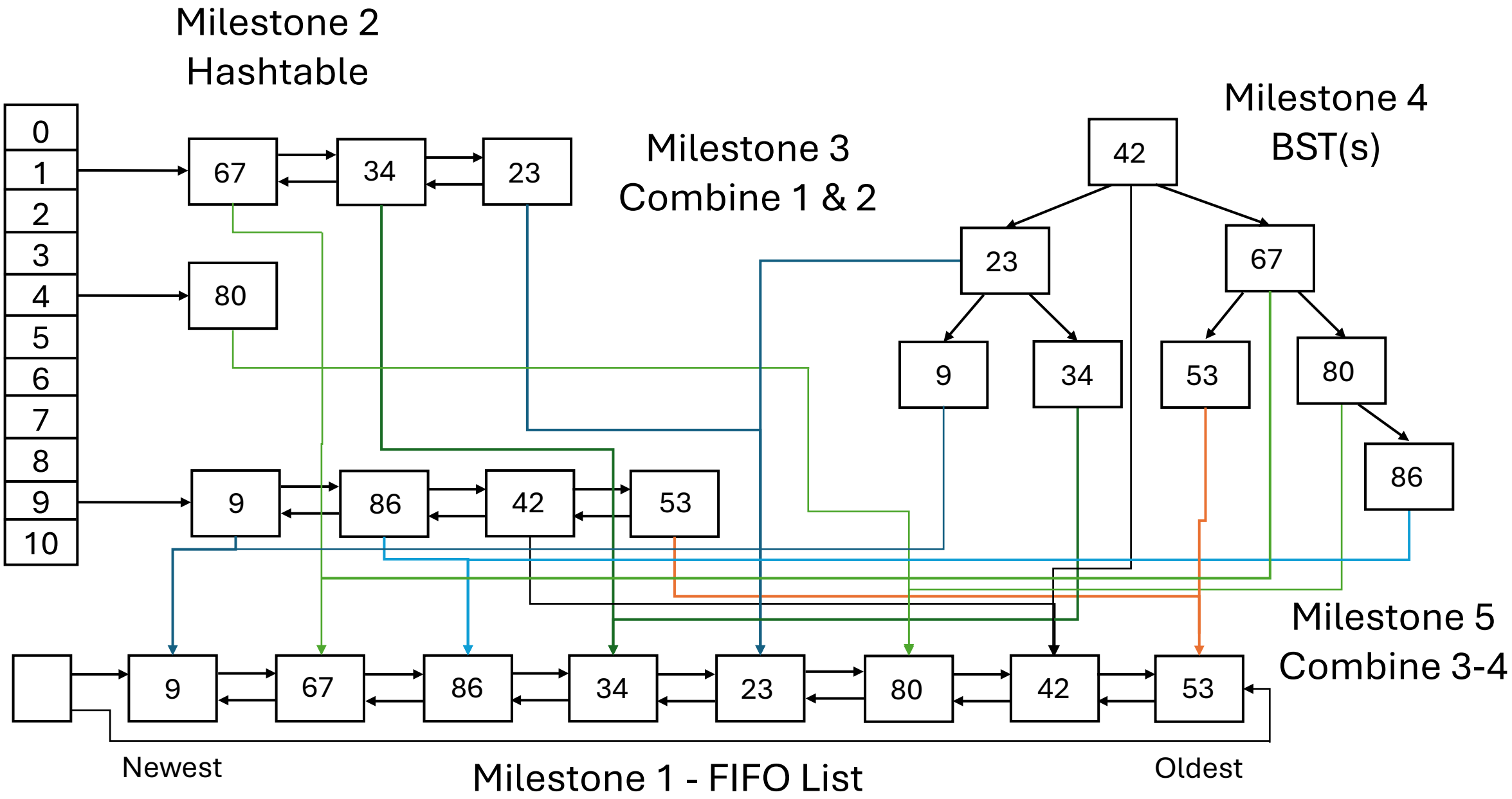
Review Week 1

- Class overview
- Project overview
- Professional Software Development

Class Overview

- Programming Methodology
  - What is Programming Methodology?
- Introduction to C++
  - Syntax, memory management, STL, C++ versus Java
- Advanced Data Structures
  - More coverage of Data Structures studied in CSC 220

Milestone Overview – Cache Manager



Professionally written code

- Effective and efficient
  - Does what it is supposed to do
  - Does not consume more CPU or memory than needed
  - Performant
  - Stable/Reliable

- Maintainability
  - Human understandable
  - Take advantage of Libraries and Frameworks
  - Documentation, Documentation, and more Documentation
  - Testing

- Modular
  - Break large problems into smaller problems
  - Larger methods are generally harder to reuse

When to use a particular Data Structure

- Go over scenarios and decide which Data Structure to use

- Scenarios will be from Job Interview Questions
  - Almost all interviews for software development have Data Structure questions
  - My personal favorites, as well as interview questions on web

- The one skill I want you to develop in this class, is the ability to analyze a problem and determine which Data Structure(s) to use to solve it.
  - Good for Job Interviews
  - But, more importantly, it’s good to know for your career

Programming Style

- Writing easy to read and effective programs is a skill

- Strive to be always write clear and understandable code
  - 90% of the cost for industry code is for maintenance
  - Clear and understandable code is maintainable code

- Follow corporate coding standards
  - Good names for variables and methods
  - Always create File and Method comment headers
  - Comment the code for the next person
  - Liberal use of whitespace

Generative AI as a Crutch

- Dependence on Generative AI can be detrimental to your skills
- ~80% of generated code is good (this will improve)
- How will you determine which 20% is not good?
- Experience is needed to determine good from not so good
- Experience comes from practice and more practice

Closing Words on Programming Methodology

"Any fool can write code that a computer can understand.

Good programmers write code that humans can understand."

Martin Fowler

Author of multiple OO and UML texts

Pseudocode

- What is pseudocode?
  - It is NOT writing code in any specific language
- Express in English, what an algorithm does
  - Avoids syntax details
- If-then, while, case can be used (almost all languages support)
- Should be understandable to a non-programmer

Review Week 2

- C++ Syntax
- Functions
- Arrays and Vectors
- Strings
- Inheritance

C++ Syntax

1. Enables input and output ----------> `#include <iostream>`
2. Opens the standard namespace ----> `using namespace std;` // syntax structure allowing grouping of data & methods
3. Begins the function **main** ----------> `int main()` // main must be global and return int
4. A comment ----------> `// Computes the volume of a sphere of a given radius.`
5. Begins body of function ----------> `{`
6. Defines a constant ----------> `const double PI = 3.14159;` // constants do not change
7. Declares a variable ----------> `double radius;` // variables begin with a letter or underscore

8. Displays a prompt to the user -------> `cout << "Enter the radius of the sphere: ";`
9. Reads **radius** ----------> `cin >> radius;`

10. Declares and computes **volume** ---> `double volume = 4 * PI * radius * radius * radius / 3;`
11. Displays results ----------> `cout << "The volume of a sphere of radius "`
12. Statement continues ----------> `<< radius << " inches is " << volume`
13. Statement continues ----------> `<< " cubic inches.\n";`
14. Normal program termination -------> `return 0;`
15. Ends body of function ----------> `} // end program`

Multi-line comments start with /* and end with */

C++ Functions

- An argument passed by reference is not copied but is accessed directly within the function.
- Which variables are passed by Value? Reference?

```
void computeMax(int x, int y, int& larger)        // void function does not return a value
{
        larger = ((x > y) ? x : y):
}       // end computeMax

int main ()
{
         int a = 0;
         int b = 0;
         int largerAB = 0;
         cin >> a >> b;
         computeMax(a, b, largerAB);
         cout << "The larger of " << a << " and " << b << " is " << largerAB << ".\n";
}
```

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Passing Arrays to a Functions

- Arrays are **always** passed by **reference** to a function.
- Common ways to pass arrays to function:
  1. void modifyArray(int arr[]) // same as #2
  2. void modifyArray(int *arr) // same as #1
  3. void modifyPointer(int*& arr) // allows you modify array elements and pointer itself; useful when you want to change memory location

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Strings

- The C++ Standard Library provides the data type string. To use this library, you include the statements

```
#include <string>
using namespace std;
```

- You can concatenate two strings to form another string by using the + operator.
- You can manipulate a portion of a string by using the method

```
substr(position, length )
```

- The first argument specifies the position of the beginning of the substring (remember that 0 is the position of the first character in the string). The second argument is the length of the substring. For example, (e.g., title = "J Perfect's Diary");

```
title.substr(2, 7)
```

- is the string “ Perfect ”.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Important Themes in Programming

- Choose identifiers that **describe** their purpose—that is, that are self-documenting. Distinguish between keywords, such as int, and user-defined identifiers.
- Use a good **indentation** style to enhance the readability of a program. The layout of a program should make it easy for a reader to identify the program's modules.
- Use a **blank line** between method or function definitions.
- Within these definitions, you should **indent** individual blocks of code. These blocks are generally—but are not limited to—the actions performed within a control structure, such as a while loop or an if statement.
- Named constants make a program easier to modify, e.g.,

```
const int NUMBER_OF_MAJORS = 202;
```

- typedef statements make a program easier to modify; if your program performs floating point computations, and you need greater precision than float (change float to double)
  - typedef float RealType; // original statement
  - typedef long double RealType; // update to this statement in one spot

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Testing and Programming

- Two kinds of testing can be applied to a system:
  - Open-box testing (**white-box** or glass-box testing) - design tests knowing how the modules work.
    - choose a method's actual arguments in a way that exercises all lines of code in the method.
    - make sure that all branches of if and switch statements are executed.
  - Closed-box testing (**black-box** or functional testing) - you know the name of the method being tested, as well as its argument list, its return type, and its operation contract.
    - develop tests across a range of valid input, and then check the output against hand-calculated results.
    - Choose arguments outside of their valid range tests how the module handles incorrect input.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Inheritance

- Inheritance allows us to reuse and extend work we have already completed and tested.
- You can think of a base class as a parent class.
  - base class = superclass = parent class
- You can think of a derived class as a child class
  - derived class = subclass = child class
- Any instance of the derived class is also considered to be an instance of the base class and can be used in a program anywhere that an instance of the base class can be used
- When the keyword public is used with the base class, any of the publicly defined methods or data fields in the base class can be used by instances of the derived class.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Overriding Base-Class Methods

- You can add as many new members to derived class as desired
- You can redefine inherited methods
  - Called overriding a base-class method.
- A method overrides a base-class method when
  - The two methods have the same name and parameter declarations
- Precede a method name with the base-class namespace indicator to tell the compiler to use the base-class version of the method
- Without a namespace indicator in front of the method name, the compiler assumes that the programmer is referring to the current class and inserts code to call that version of the method

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Virtual Methods, Abstract Classes

- Using keyword **virtual** in front of the prototype
  - Tells the C++ compiler that the code this method executes is determined at runtime
  - Allows an application using our class to take advantage of polymorphism when the methods are invoked
- Pure virtual method
  - Virtual method that has no implementation
- Abstract class
  - Has at least one pure virtual method

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Review Week 3

- Memory Management
  - Many differences between C++ and Java
- Memory Leaks in C++
- C++ Inheritence

Pointers in C++

- In Java, an object is *always* referred to via a pointer variable that points to it; on the other hand, a primitive type value is always referred to directly, and *never* referred to via a pointer variable that points to it
- In C++, you have a choice whether to refer to an instance of any type directly, or via a pointer
- In particular, C++, you can create a pointer that points to any type you want (even another pointer), and manipulate the values stored in the pointers, and in the memory they point to, in absolute detail
- This is an extremely powerful feature of C and C++, and it exposes some of the implementation details of pointers that are abstracted in Java

Making pointers point

- The "address-of" prefix operator **&** applied to a variable returns the memory address of the variable

```
int a=5, b= -999;

int* pt1 = &a; //   initialize pt1 to point to a
```

address	*memory cell*	*identifier*
512000	5	**a**
512004	-999	**b**
512008	**512000**	**pt1**

What is the output?

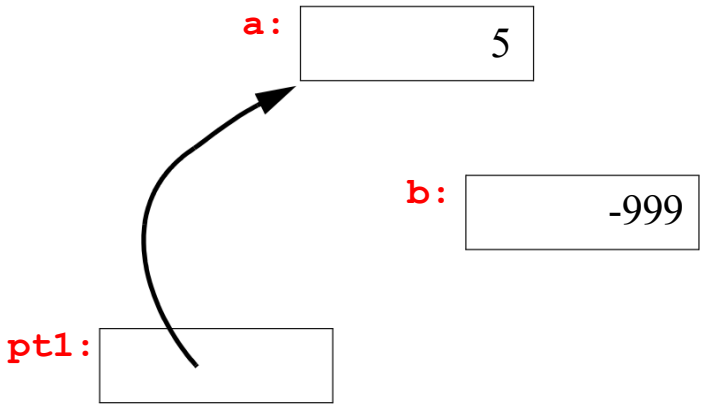
```
cout << "a is:" << a;            // displays: a is: 5
cout << " &a is: " << &a;        // displays: &a is:  512000
cout << " &pt1 is: " << &pt1;    // displays: &pt1 is:  512008
cout << " *pt1 is: " << *pt1;    // displays: *pt1 is:  5
cout << " pt1 is: " << pt1;      // displays: pt1 is:  512000
```

An abstract picture of memory

- A useful abstract way to draw the contents of memory after those statements execute is shown here. The exact value of the address in the pointer is not shown. Instead, the fact that the pointer named `pt1` points to the variable named `a` is shown with an arrow:

```
int a=5, b= -999;

int* pt1 = &a;     // initialize pt1 to point to a
```



- Note: this picture is impossible in Java, which does not allow pointers to point directly to primitive type variables

Dereferencing pointers

- The "dereferencing" prefix operator * applied to a pointer value "goes to" the location in memory pointed to by the pointer

```
int a=5, b= -999;

int* pt1 = &a;     // declare and initialize pt1 to point to a

cout << *pt1;      // print the value of what pt1 points to:5

*pt1 = 10;         // assign a value to what pt1 points to

cout << a;         // print the value of a, which is...10
```

- In C++ you can have pointers to anything! For example you can create a pointer to a pointer to an int:

```
int ** pp = &pt1; // declare and initialize pp to point to pt1

**pp = 20;  // assign a value to what pp points to points to...!

cout << a   // print the value of a, which is...20
```

Pitfalls with arrays and pointers in in C++

- With a declaration like

  ```
  int a[100];
  ```

  expressions like `a[-33]` or `a[101]` or `*(a + 10000)` are very dubious, to say the least

- However, these are not compiler errors, and may not even give you runtime errors (unless they involve accessing protected memory)

- The power of pointer arithmetic can hurt you, if you're not careful...

- For these reasons, though C-style arrays are available in C++, it is advisable to use data structures from the Standard Template Library (STL) such as `std::vector` instead of basic arrays

Strings as arrays of chars in C++

- As in C, an array of char elements with an element `'\0'` used to terminate the string can be used as a string in C++
  - `'\0'` is the null character, an 8-bit integer with value 0
  - to contain a string, a char array must contain a null character
  - the null character is used to determine the end of the string contained in the array

- A null-terminated array of char can be initialized with a string literal:

```
char s[] = "hello";
```

  - this creates an array `s` with 6 elements, not 5!
  - the null character is included, as the last element of `s`
  - this is equivalent to

```
char s[] = {'h','e','l','l','o','\0'};
```

or

```
char s[6] = "hello";
```

Dealing with strings in C++

- Since in C++ a null-terminated char array (like any array) does not keep information about its own length, extra checking must be done to make sure all accesses to the string are within bounds

- This checking is tedious to do, but if not done, leads to serious bugs and exploitable code
  - Most software security problems are due to unchecked array bounds violations (buffer overflows) in C and C++ programs!

- Again, in C++, it is advisable to use data structures from the Standard Template Library (STL): instead of null-terminated char arrays, use `std::string` for strings!

C++ Destructors

- Destructors
  - destroys an instance of a class when the object's lifetime ends
  - A class has one destructor
  - Automatically called when an object goes out of scope or is deleted
  - For many classes, you can omit the destructor. In such cases, the compiler generates a destructor for you.
  - If the class itself creates an object on the free store (heap) by using the new operator, it is a safe and secure programming practice to implement a destructor to ensure that the memory for that object is freed (cover in Interlude 2)

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Preprocessor directives

- Preprocessor directives
  - Provides a way to prevent the compiler from reading the class definition a second time, if class definition is included multiple times:
    - #ifndef PLAIN_BOX // If PLAIN_BOX is not defined, then …”
    - #define PLAIN_BOX // defines the name PLAIN_BOX
    - …
    - #endif // PLAIN_BOX

If the compiler had not defined the name PLAIN_BOX, it would process the code that follows until it reached the #endif directive at the end of the file.

If another file includes the class definition of PlainBox, the name PLAIN_BOX will already have been defined. That file’s #ifndef directive will cause the preprocessor to skip any of the code that follows, so the compiler will not see the same class definition more than once.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Inheritance

- Inheritance allows us to reuse and extend work we have already completed and tested.
- You can think of a base class as a parent class.
  - base class = superclass = parent class
- You can think of a derived class as a child class
  - derived class = subclass = child class
- Any instance of the derived class is also considered to be an instance of the base class and can be used in a program anywhere that an instance of the base class can be used
- When the keyword public is used with the base class, any of the publicly defined methods or data fields in the base class can be used by instances of the derived class.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ derived class

- has the data fields and methods of the base class in addition to the members it defines,
- it can access only publicly defined members from the base class
- can also revise any inherited public method.
- contains copies of inherited data fields,
- does not contain a copy of the code for inherited methods
- does not inherit constructors from parent class, but it can use it or other parent method to initialize the private data fields

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Inheritance

```
#include <iostream>
#include <string>
using namespace std;

class Vehicle {
  public:
    string brand = "Chevrolet";                    // Default brand is Chevrolet
    void playStereo() {
      cout << "\nBlasting it! \n";                 // Simulates playing the stereo
   }
};

// The Car class is derived from the Vehicle class and adds attributes and behaviors specific to a car.
class Car: public Vehicle {
  public:
    string model = "Corvette";   //  Default model is Corvette
};

int main() {
  Car myCar;                                       // Create a Car object named myCar
  cout << myCar.brand + " " + myCar.model; // Output the brand and model of the car
  myCar.playStereo();                              // Call the playStereo function from the Vehicle class
  return 0;                                        // Return 0 to indicate successful execution
}
```

What is the console output?

Chevrolet Corvette
Blasting it!

Overriding Base-Class Methods

```
#include <iostream>
#include <string>
using namespace std;

class Vehicle {
  public:
    string brand = "Chevrolet";                    // Default brand is Chevrolet
    void playStereo() {
      cout << "\nPlay at regular volume\n";     // Simulates playing the stereo
   }
};

// The Car class is derived from the Vehicle class and adds attributes and behaviors specific to a car.
class Car: public Vehicle {
  public:
    string model = "Corvette";   //  Default model is Corvette
    void playStereo() {
      cout << "\nBlasting it! \n" ;
    }
};

int main() {
  Car myCar;                                       // Create a Car object named myCar
  cout << myCar.brand + " " + myCar.model; // Output the brand and model of the car
  myCar.Vehicle::playStereo();                     // Call the playStereo function from the Vehicle class
  myCar.playStereo();                              // Call the playStereo function from the Car class
  return 0;                                        // Return 0 to indicate successful execution
}
```

What is the console output?

```
Chevrolet Corvette
Play at regular volume

Blasting it!
```

C++ Review

```
int foo(int i) {
   int j = i + 1;
   C* x;
   x = new C();
   x->setA(j);
   j = x->getA();
   delete x;
   return j;
 }
```

1. Which variables are Automatic?
2. Where are Automatic variables allocated?

Virtual Methods and Polymorphism

- Allow compiler to perform the late binding necessary for polymorphism
  - Declare methods in base class as *virtual*, to enable late binding for polymorphism; indicates methods can be overridden
  - Create variables in the free store (heap), and use pointers to reference them
  - Change function to accept that pointer
  - Free the memory in heap; delete and assign to nullptr

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Early versus Late Binding

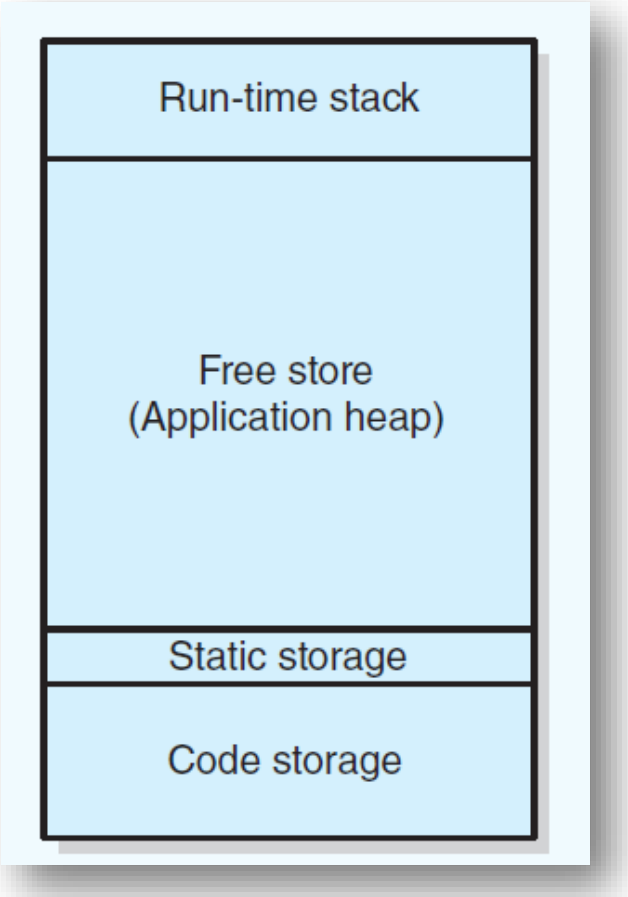
- Use early binding for better performance and when you don't need polymorphism.
- Use late binding (virtual functions) for polymorphism, and need to ensure the derived class method is called at runtime, even when using a base class pointer or reference
- declare function virtual in the base class, which allows compiler to determine appropriate function to call at runtime, based on object type

https://www.geeksforgeeks.org/early-binding-late-binding-c/

Pointers and Program's Free Store

When a C++ program begins execution, the operating system sets aside memory for

- Run-time stack,
- Code storage – store your code, or text storage,
- Static storage – store any global variables and static variables,
- Heap, or free store, which a programmer can use to store data



- FIGURE C2-2 Sample program memory layout

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Deallocating Memory

- When memory to which pointer variable points is no longer needed
  - Deallocate it by using delete operator.
  - Then set pointer variable to *nullptr*
- Otherwise dangling pointer exists
  - It would still contain address of object that was deallocated.
  - Can be source of serious errors.
- C++ memory leak tools: https://learn.microsoft.com/en-us/cpp/c-runtime-library/find-memory-leaks-using-the-crt-library?view=msvc-170

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Avoiding Memory Leaks

- Memory leaks occur when
  - An object has been created in the heap (free stores), but
  - Program no longer has a way to access

```
void myLeakyFunction(const double& someItem)
{
   ToyBox<double>* someBoxPtr = new ToyBox<double>();
   someBoxPtr->setItem(someItem);
}  // end myLeakyFunction
```

We have memory leak, since someItem and the pointer someBoxPtr are both stored on the stack, which are destroyed after the function ends.

LISTING C2-1 Poorly written function that allocates memory in the free store

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Avoiding Memory Leaks

- The best option for preventing a memory leak is to not use a function to return a pointer to a newly created object
- You should define a class that has a method for this task
- Class takes responsibility for deleting the object in the heap (free store) and ensures that there is no memory leak. At a minimum, such a class will have three parts:
  1. a method that creates the object in the free store,
  2. a data field that points to the object, and
  3. a method—the destructor —that deletes the object when the class instance is no longer needed

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Avoiding Memory Leaks

- Destructors
  - Every C++ class has a destructor that has the same name as the class, but is preceded by the tilde (~) character.
  - For objects that are local variables, the destructor is called when the activation record containing the object is removed from the run-time stack.
  - If the object was created using new and is stored in the free store, the destructor is called when the client uses the operator delete to free the memory allocated to the object.
  - Often, the compiler-generated destructor is sufficient for a class, but if the class itself creates an object on the free store (heap) by using the new operator, it is a safe and secure programming practice to implement a destructor to ensure that the memory for that object is freed

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

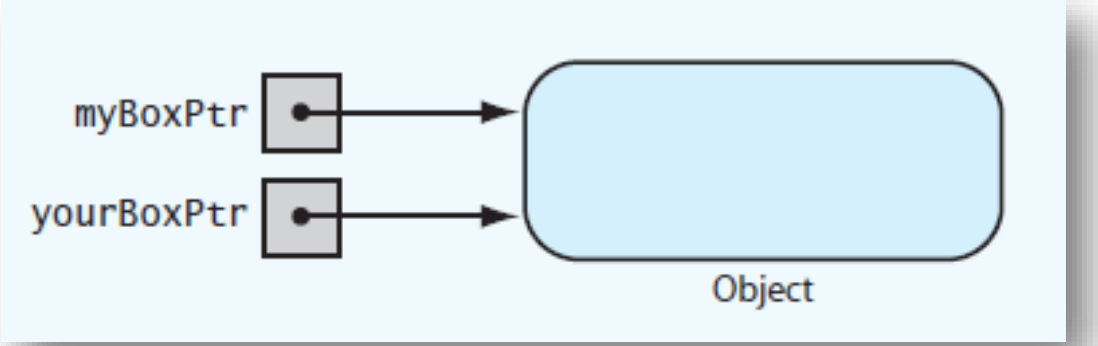
Avoiding Dangling Pointers

If you have two (2) or more pointers pointing to the same object (alias), and

- you delete the first pointer to the object, and
- attempt to execute the method with the second pointer, will result in a runtime error, since the second pointer is a dangling pointer

o MagicBox<string>* myBoxPtr = new MagicBox<string>();

o MagicBox<string>* yourBoxPtr = myBoxPtr;



Suppose we then execute:

```
delete myBoxPtr;
myBoxPtr = nullptr ;
yourBoxPtr->getItem();
```

- Program aborts because yourBoxPtr still references the object's location in the heap, even though the object no longer exists
- yourBoxPtr is a dangling pointer and the program aborts when we try to call a method on that object

- FIGURE C2-7 Two pointers referencing (pointing to) the same object

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Avoiding Dangling Pointers

- Set pointer variables to nullptr either initially or when you no longer need them.
- If a class has a pointer variable as a data field, the constructor should always initialize that data field, to point either to an object or to nullptr.
- Test whether a pointer variable contains nullptr before using it to call a method.
- Try to reduce the use of aliases in your program.
- Do not delete an object in the free store (heap) until you are certain that no other alias needs to use it.
- Set all aliases that reference a deleted object to nullptr when the object is deleted.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Aspects of Object-Oriented Solution

Principles of object-oriented programming

- Encapsulation: Objects combine data and operations; hides inner details
- Inheritance: Classes inherit properties from other classes; allows class reuse
- Polymorphism: Objects determine appropriate operations at execution time.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Cohesion

- Each module should perform one well-defined task - cohesive
- Benefits
  - Well named, self-documenting, e.g., sort
  - Easy to reuse
  - Easier to maintain
  - More robust; less likely to be impacted by change

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Coupling

- Measure of dependence among modules
- Dependence
  - Sharing data structures or calling each other's methods
- Modules should be loosely coupled
  - Highly coupled modules should be avoided
- Benefits of loose coupling in a system
  - More adaptable to change
  - Easier to understand
  - Increases reusability
  - Has increased cohesion

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Abstraction

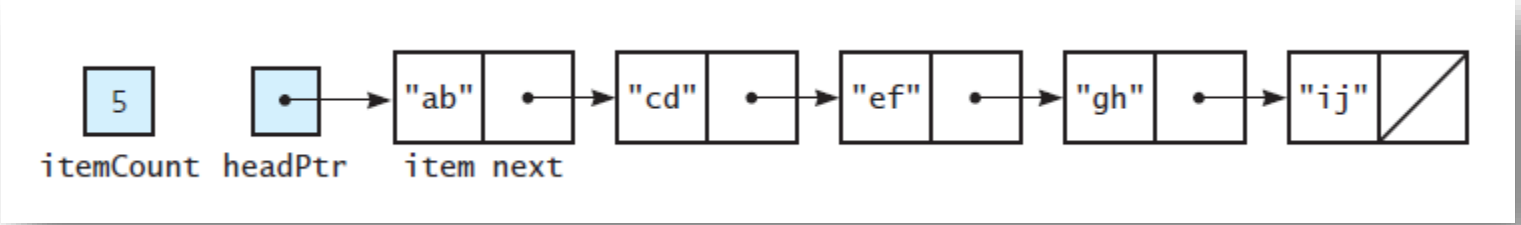
- Separate purpose of a module from its implementation
  - For example, you should be able to change the sorting algorithm in an implementation without affecting the rest of the solution
- Specifications do not indicate how to implement
  - Able to use without knowing implementation
- Abstraction helps identify details that should be hidden from public view
  - Ensured no other module can tamper with these hidden details.
- Isolation of the modules cannot be total, however
  - Client must know what tasks can be done, how to initiate a task

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Review Week 4

- ADT Bag
- ADT Dictionary

A Link-Based Implementation of the ADT Bag



- FIGURE 4-6 A link-based implementation of the ADT bag

```
+getCurrentSize(): integer
+isEmpty(): boolean
+add(newEntry: ItemType): boolean
+remove(anEntry: ItemType): boolean
+clear(): void
+getFrequencyOf(anEntry: ItemType): integer
+contains(anEntry: ItemType): boolean
+toVector(): vector
```

Bag operations, given in UML notation

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

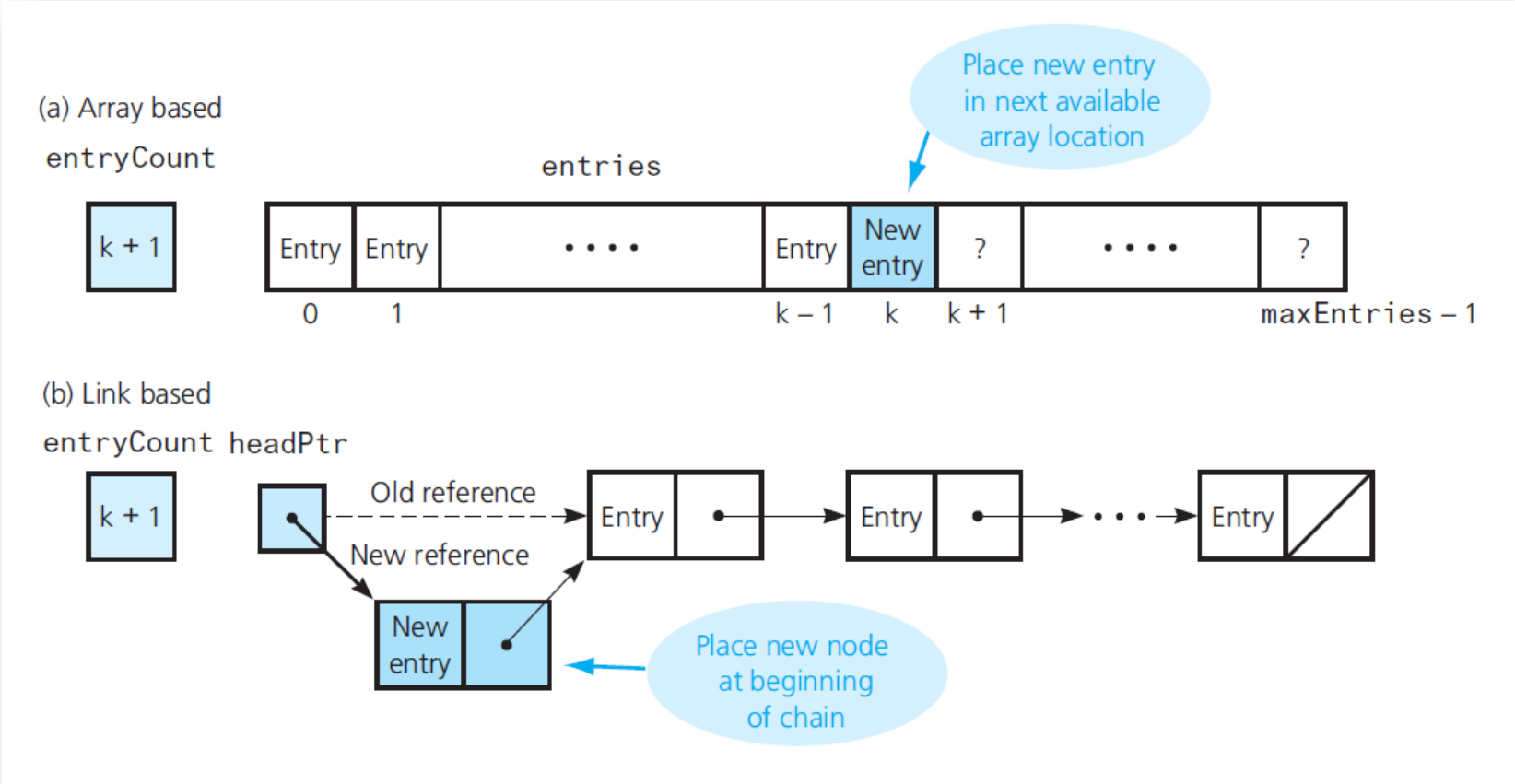
The ADT Dictionary

```
Dictionary
---------------------------------------------------------------
---------------------------------------------------------------
+isEmpty(): boolean
+getNumberOfEntries(): integer
+add(searchKey: KeyType, newValue: ValueType): boolean
+remove(targetKey: KeyType): boolean
+clear(): void
+getValue(targetKey: KeyType): ValueType
+contains(targetKey: KeyType): boolean
+traverse(visit(value: ValueType): void): void
```

- FIGURE 18-2 UML diagram for a class of dictionaries

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Three Scenarios



- FIGURE 18-6 Addition for unsorted linear implementations

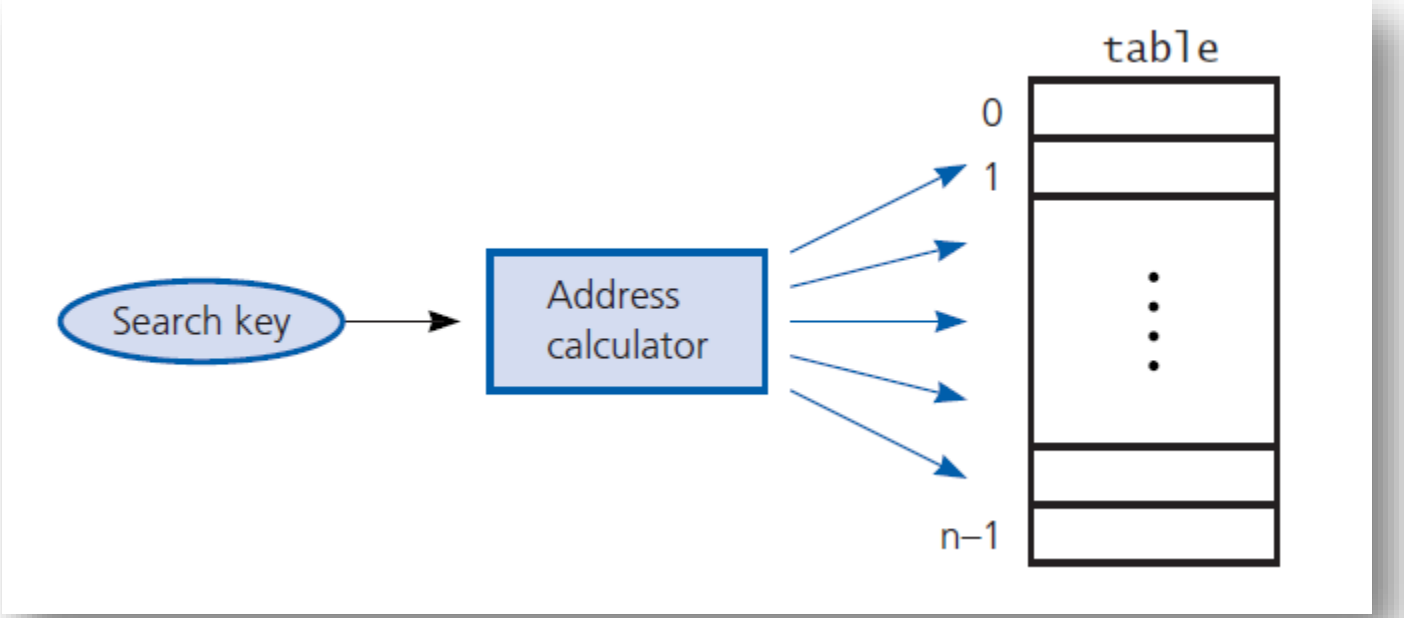
© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Hashing as a Dictionary Implementation

- Situations occur for which search-tree implementations are not adequate.
- Consider a method which acts as an “address calculator” which determines an array index
  - Used for add, getValue, remove operations
- Called a hash function/hash code
  - Tells where to place item in a hash table

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Hashing as a Dictionary Implementation



- FIGURE 18-9 Address calculator

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Hashing as a Dictionary Implementation

- Perfect hash function
  - Maps each search key into a unique location of the hash table
  - Possible if you know all the search keys
- Collision occurs when hash function maps more than one entry into same array location
- Hash function should
  - Be easy, fast to compute
  - Place entries evenly throughout hash table

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Hash Functions

- Sufficient for hash functions to operate on integers – examples:
  - Select digits from an ID number
    - Generally uneven distribution
  - Folding – add digits, sum is the table location
  - Modulo arithmetic $h(x) = x \bmod tableSize$
    - Simple, effective
    - Use prime number for table size
  - Convert character string to an integer – use ASCII values
    - Address potential overflow

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

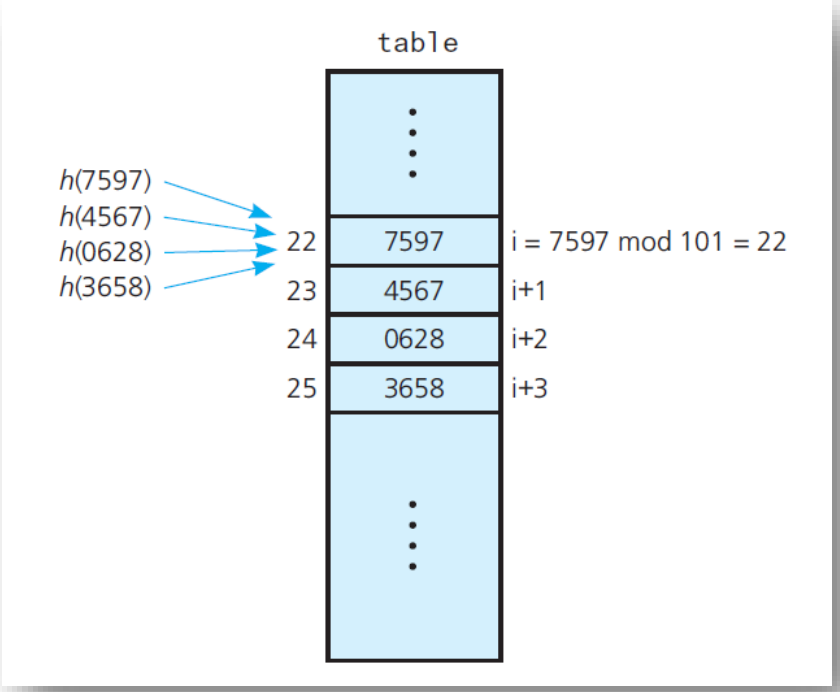
Resolving Collisions with Open Addressing

- Open addressing

  "Open" means to look for an open slot
  - Linear probing
  - Quadratic probing
  - Double hashing
  - Increase size of hash table

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Resolving Collisions with Open Addressing



Linear probing

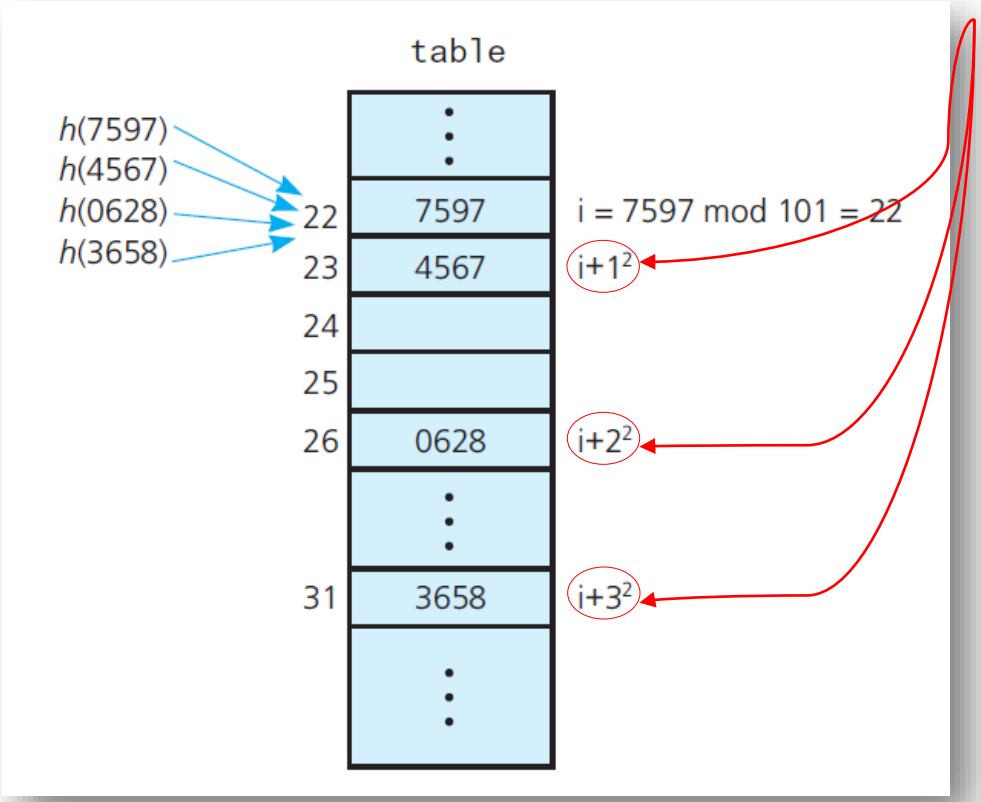
- Add - search the hash table, starting from the original hash location; if table[h(searchKey)] is occupied, check table[h(searchKey)+1] , table[h(searchKey)+2], … until you find an available location.
- Delete –place table location into one of three states:
  - occupied (currently in use),
  - empty (has not been used), or
    - Modify addItem to insert at this location
  - removed (was once occupied but is now available)
    - Modify getItem to continue probing
    - Modify addItem to insert at this location

- Items tend to cluster together in the hash table, when using this approach - primary clustering.
- Clusters can get close to one another and, merge into a larger cluster.
- Large clusters tend to get even larger

- FIGURE 18-11 Linear probing with h(x) = x mod 101

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Resolving Collisions with Open Addressing



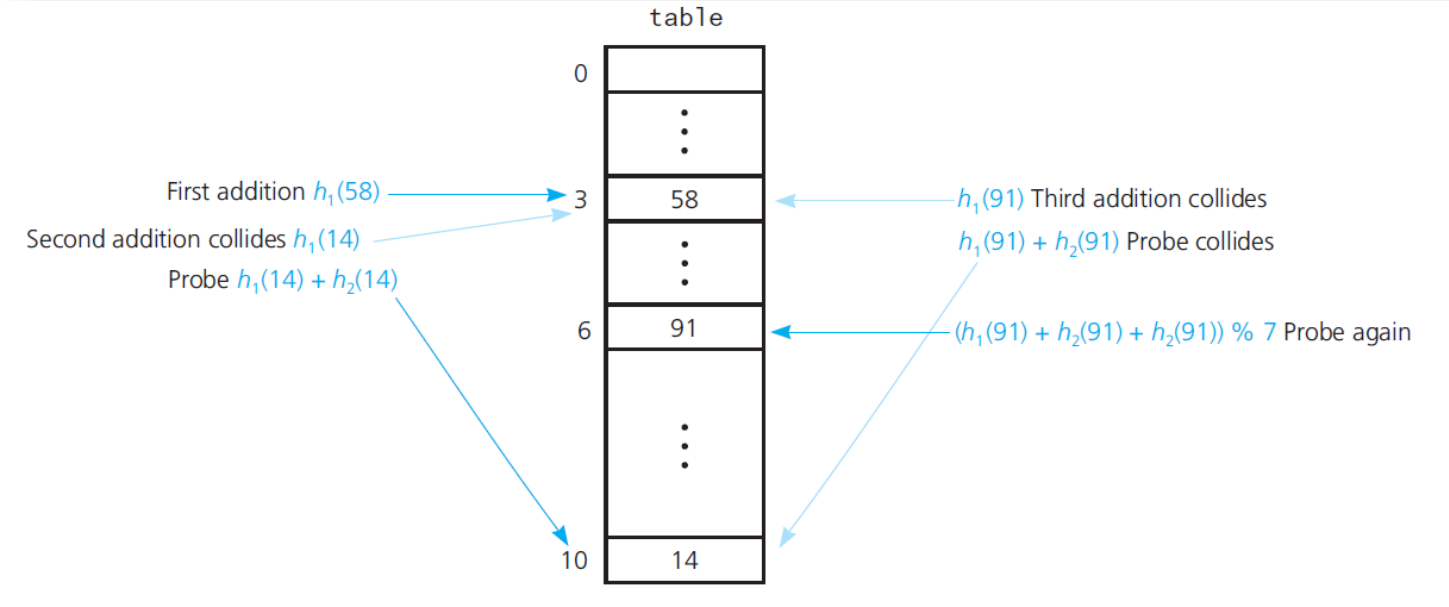
Quadratic probing.

- You can virtually eliminate primary clusters by adjusting the linear probing scheme.
- When two items hash collide, quadratic probing uses the same probe sequence for each item.
- The resulting phenomenon—called secondary clustering —delays the resolution of the collision.
- Although the research of quadratic probing remains incomplete, it appears that secondary clustering is not a problem.

- FIGURE 18-12 Quadratic probing with h(x) = x mod 101

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Resolving Collisions with Open Addressing



If key = 58,

- h1 hashes to location 3 (58 mod 11), and
- h2 indicates that the probe sequence should take steps of size 5 (7 – 58 mod 7).
- In other words, the probe sequence will be 3, 8, 2 (wraps around), 7, 1 (wraps around), 6, 0, 5, 10, 4, 9.

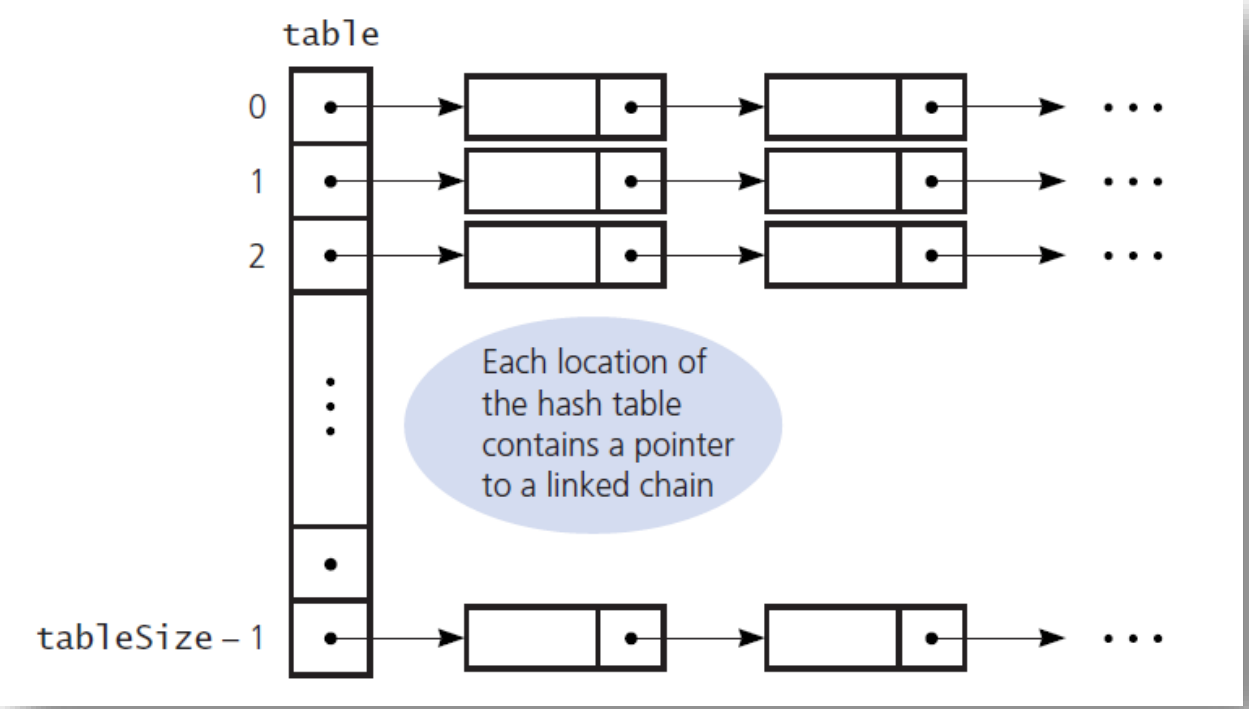
Double hashing - defines key-dependent probe sequences.

- searches the hash table in a linear order, starting at the location h1(key),
- second hash function h2 determines the size of the steps taken
  - h2(key) <> 0
  - h2 <> h1
- For example, let h1 and h2 be the primary and secondary hash functions defined as
  - h1 (key) = key mod 11 // 11 is table size and prime number
  - h2(key) = 7 - (key mod 7) // visits all the table locations (table size and step are prime)

- FIGURE 18-13 Double hashing during the addition of 58, 14, and 91

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Resolving Collisions - Table Restructure



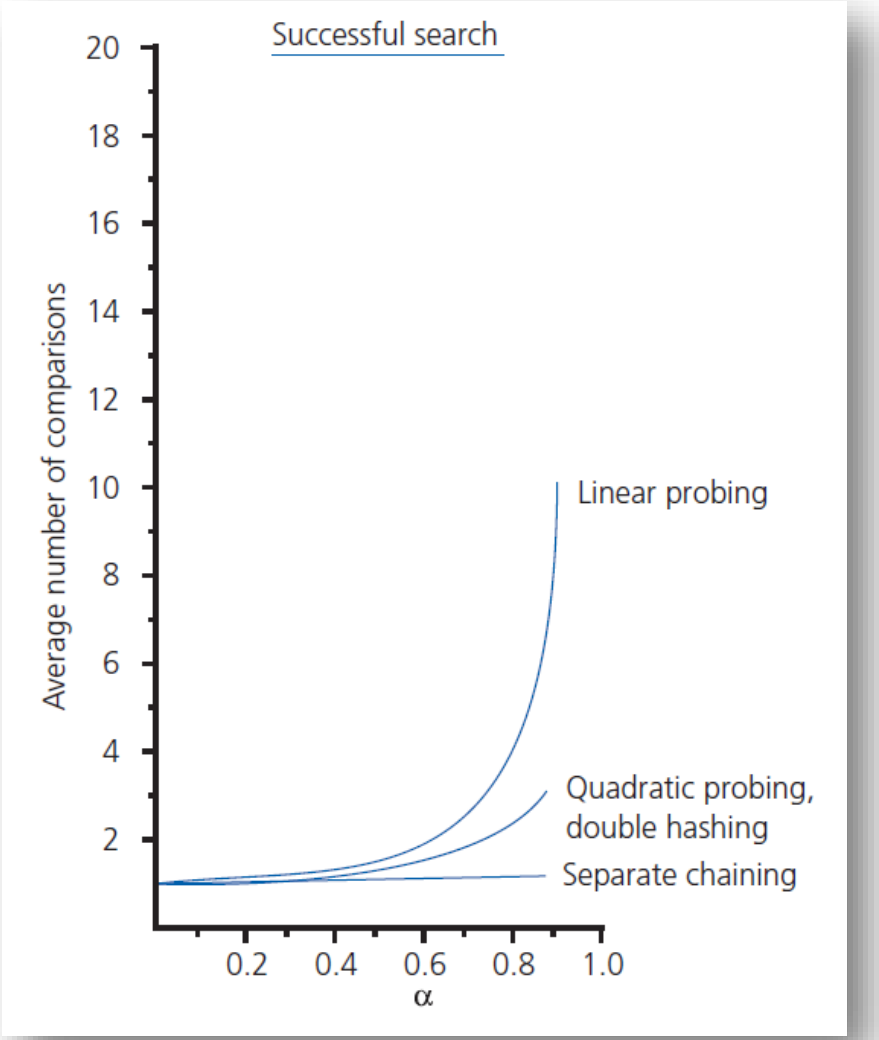
Separate chaining. Design the hash table as an array of linked chains

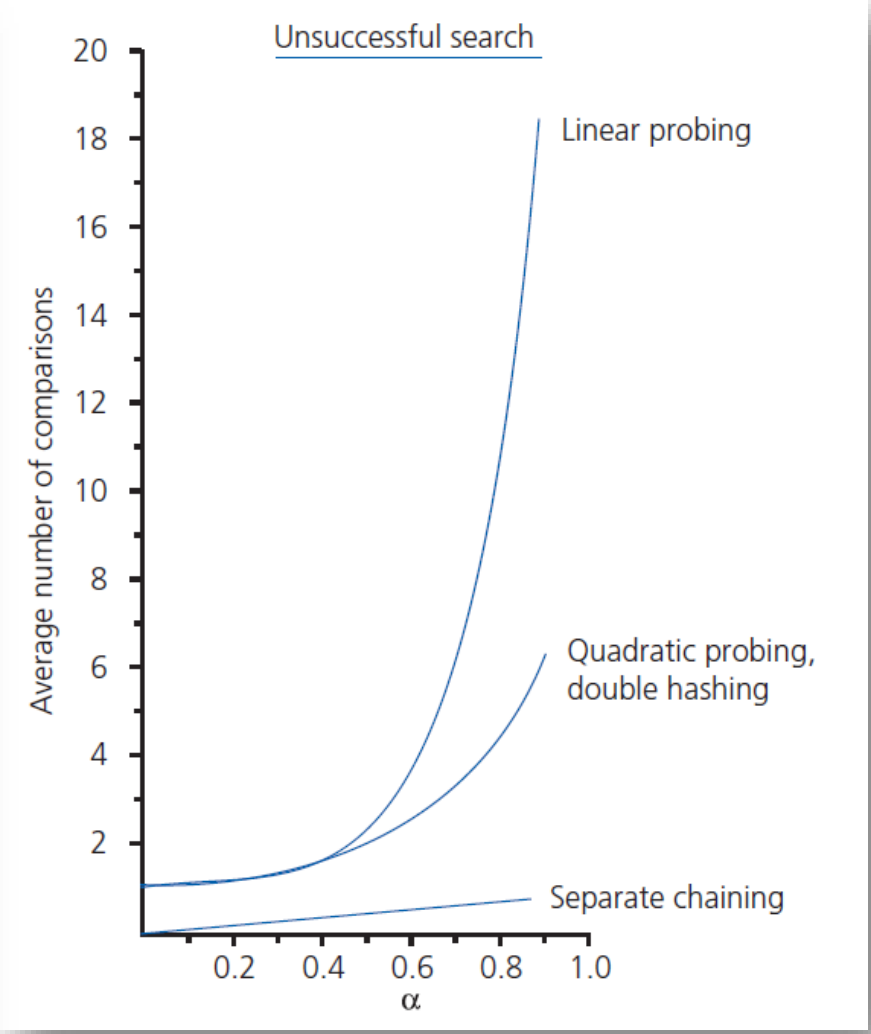
- Each entry table[i] is a pointer to a chain of linked nodes containing the items that the hash function has mapped into location i
- The size of the dictionary is dynamic and can exceed the size of the hash table, because each linked chain can be as long as necessary.
- The most time-efficient collision-resolution scheme.

- FIGURE 18-14 Separate chaining

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Efficiency of Hashing





- FIGURE 18-15 The relative efficiency of four collision-resolution methods

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

What Constitutes a Good Hash Function?

- Is hash function easy and fast to compute?
- Does hash function scatter data evenly throughout hash table?
- How well does hash function scatter random data?
- How well does hash function scatter *non*-random data?

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Review Week 5

- UML
- Recursion
- Exceptions
- Multiple Inheritance

The Unified Modeling Language

- The Unified Modeling Language, or UML, is a modeling language used to express object-oriented designs
- A use case is a set of textual scenarios or stories that describe the proposed solution, but does not describe how the system works
- A use case is not object-oriented, but it is an important part of the analysis of the problem.
- A single use case usually has several scenarios.
- The main success scenario (or “happy path”) describes how the system satisfies the goals of the user when everything goes smoothly.
- A scenario is written from the perspective of the user of the system.
- Alternate scenarios should be written only for exception cases.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The UML Sequence Diagram

- The UML sequence diagram is a design document that gives guidance on several levels to the programmer implementing a solution
  - Shows which objects collaborate to solve a particular use-case scenario
  - Shows the synchronous nature of the messages that are passed between the objects in the scenario, providing the order in which tasks are accomplished
  - Shows which object is responsible for what task or tasks by handling a particular message
  - Provides a visual representation of the flow of messages sent between objects during a single scenario

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The UML Sequence Diagram

Show objects left to right

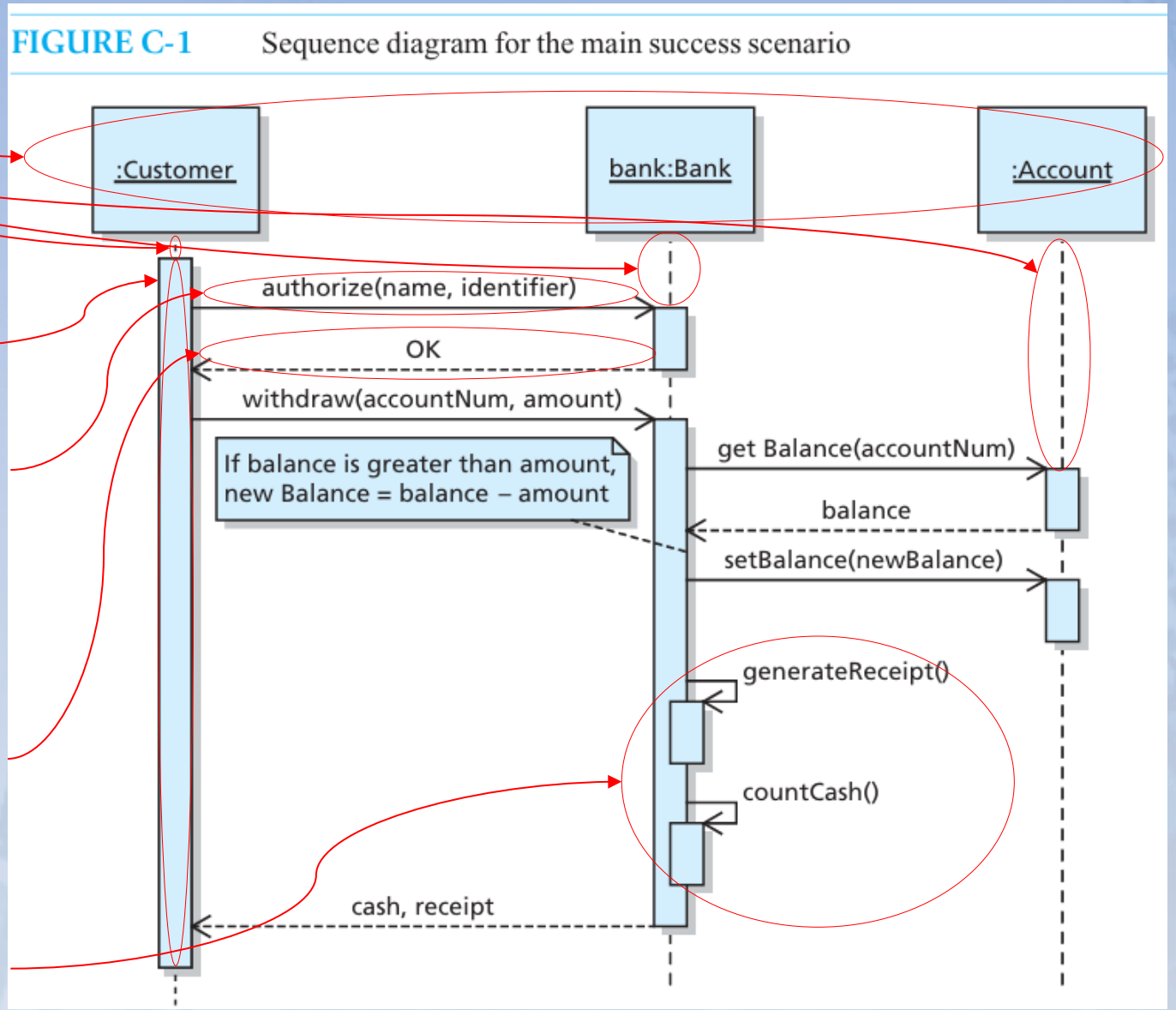
Each object has a lifeline (vertical dashed line)

Each lifeline can have 1 or more activation bars (open box on lifeline)

To send a message between objects, draw a solid arrow from the activation bar of the calling object to the top of a new activation bar on the lifeline of the called object, with message and parameters

Task completes and returns (dashed arrow w/ variable)

Objects may send messages to itself



© 2017 Pearson Education, Hoboken, NJ. All rights reserved

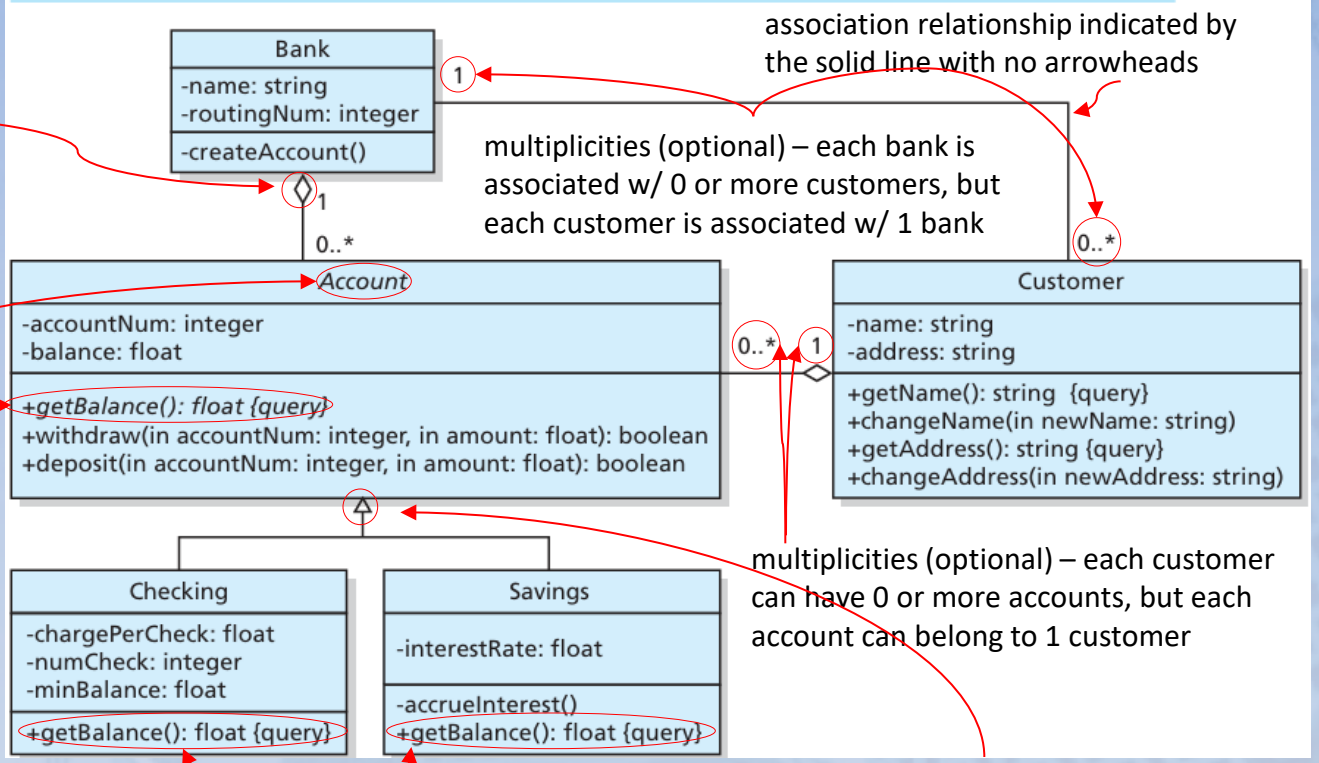
The UML Class Diagram

5 classes: 1) Bank, 2) Customer, 3) Account, 4) Checking and 5) Savings

- Bank and Account classes have **aggregation** (**containment**) relationship
- The open diamond head denotes containing class; banks live longer than accounts
- Filled in diamond head is a **composition** relationship, where lifetimes are the same

- Account class name is in italic, which denotes an abstract base class
- The getBalance operation in the Account class is also in italic. This indicates that getBalance is not implemented in the Account class but must be implemented within the descendant class.

FIGURE C-4 A UML class diagram of a banking system



association relationship indicated by the solid line with no arrowheads

multiplicities (optional) – each bank is associated w/ 0 or more customers, but each customer is associated w/ 1 bank

multiplicities (optional) – each customer can have 0 or more accounts, but each account can belong to 1 customer

- Checking and Savings classes each implement the inherited getBalance operation.
- This operation will take on polymorphic behavior when called from a parent class object.

- The Account class is related to the Checking and Savings classes via **generalization** (inheritance) relationship, indicated by the open triangular arrowhead
- Checking and Savings inherit the attributes and operations of the Account class.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

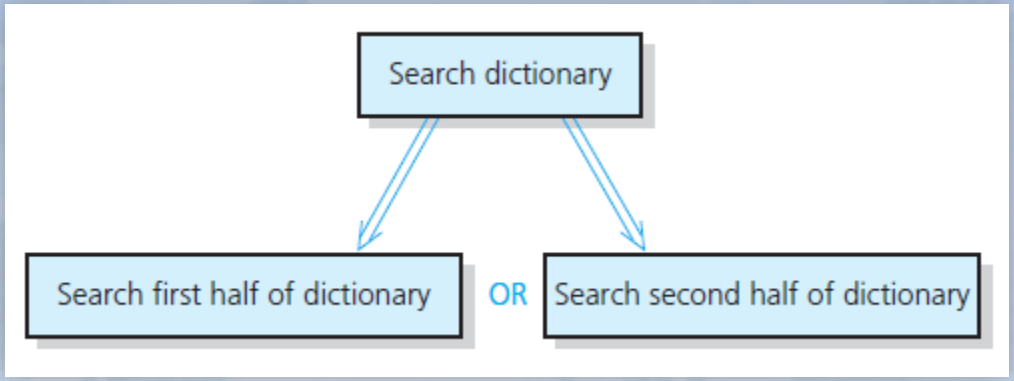
Recursive Solutions

- A recursive function calls itself
- Each recursive call solves an identical, but smaller, problem
- Test for base case enables recursive calls to stop
- Eventually, one of smaller problems must be the base case
- Some recursive solutions are impractical because they are so inefficient
- Can provide elegantly simple solutions to problems of great complexity

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Recursive Solutions

- Recursion breaks problem into smaller identical problems
  - An alternative to iteration
- FIGURE 2-1 A recursive solution



- This strategy is called divide and conquer.
- Solve the dictionary search problem by first dividing the dictionary into two halves and then conquering the appropriate half.
- Solve the smaller problem by using the same divide-and-conquer strategy.
- Dividing continues until you reach the base case.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Recursion and Efficiency

- Factors that contribute to inefficiency
  - Overhead associated with function calls
  - Some recursive algorithms inherently inefficient
- Keep in mind
  - Recursion can clarify complex solutions … but …
  - Clear, efficient iterative solution may be better

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Throwing Exceptions

Syntax

```
throw ExceptionClass(stringArgument);
```

logic_error exceptions (or one of its derived classes) represent errors in program logic (e.g., unmet precondition or failure to satisfy a postcondition, that you could detect before the program executes).

- test for an error or unusual condition using an if statement; throw an exception containing a message to our client, if error occurs
- To restrict the exceptions, you can include a throw clause in its header, informing you that the function might throw an exception.
- Including a throw clause in a function specification ensures that the function can throw only those exceptions. An attempt by the function to throw any other exception will result in a runtime error.
- Omitting a throw clause allows a function to throw any exception
- You can handle only exceptions of the class logic_error and its derived classes. To handle an exception, you write code that consists of two pieces:
    - **try** block - contains the statements that might cause or throw an exception
    - one or more **catch** blocks that follow the try block

runtime_error exceptions (or one of its derived classes) are thrown by errors that are detected at runtime (e.g., division by zero or numeric overflow)

- program will halt, just like an assertion fail
- can contain a message that is displayed to the user giving details about the error and the reason the program halted

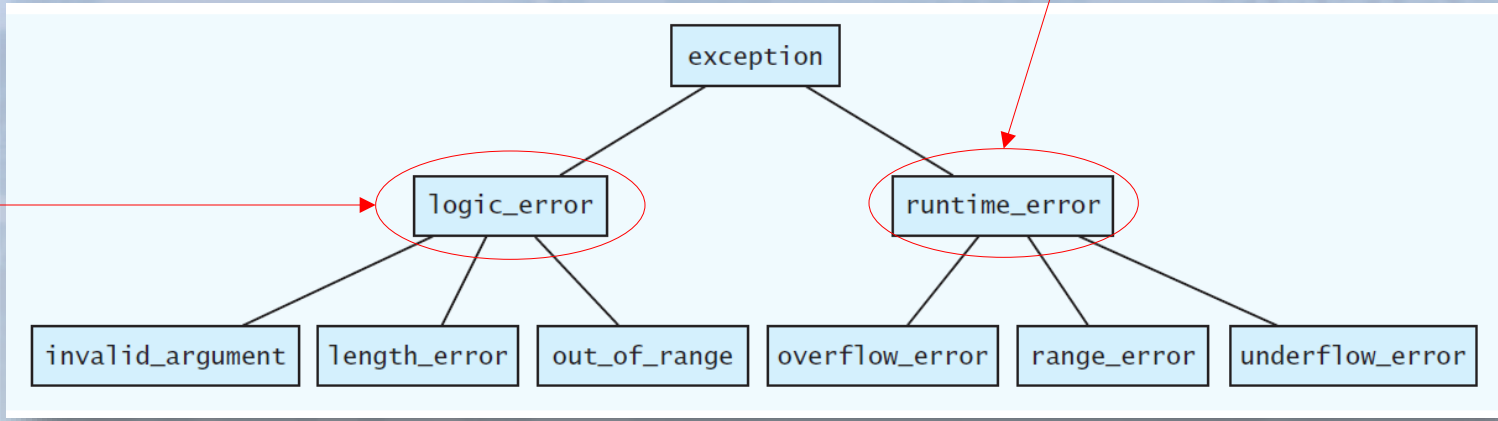


FIGURE C3-1 Hierarchy of C++ exception classes

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Handling Exceptions

- Code for handling exception
  - try block – contains statements that might cause or throw an exception
  - catch block – immediately follows try block with code to react to or catch a particular type of exception

```
try
{
    < statement(s) that might throw an exception >
}
catch (ExceptionClass  identifier)
{
    < statement(s) that react to an exception of type ExceptionClass >
}
```

General syntax for a try block followed by one catch block

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Handling Exceptions

- If no exception occurs and try block completes
  - Execution continues with statement after catch block
- If statement within try block causes exception of type specified in catch block
  - Remainder of try block abandoned
  - Execution transfers to statements in catch block
  - After catch block statements finish, execution jumps to statement after last catch block

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Handling Exceptions

- The syntax for catch block resembles that of a function definition
  - Specifies type of exception, and an identifier
  - The catch block parameter provides name for caught exception
- Steps taken in catch block vary
  - Simple message
  - Elaborate update of variables, retry of offending function

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Programmer-Defined Exception Classes

```
#include <stdexcept>
#include <string>
class TargetNotFoundException: public std::exception
{
public :
   TargetNotFoundException(const std::string& message = "")
                          : std::exception("Target not found: " + message)
   {
   } // end constructor
}; // end TargetNotFoundException
```

```
throw TargetNotFoundException (target + " not found in a box!");
```

Example – constructor provides way for throw statement to identify condition of exception.

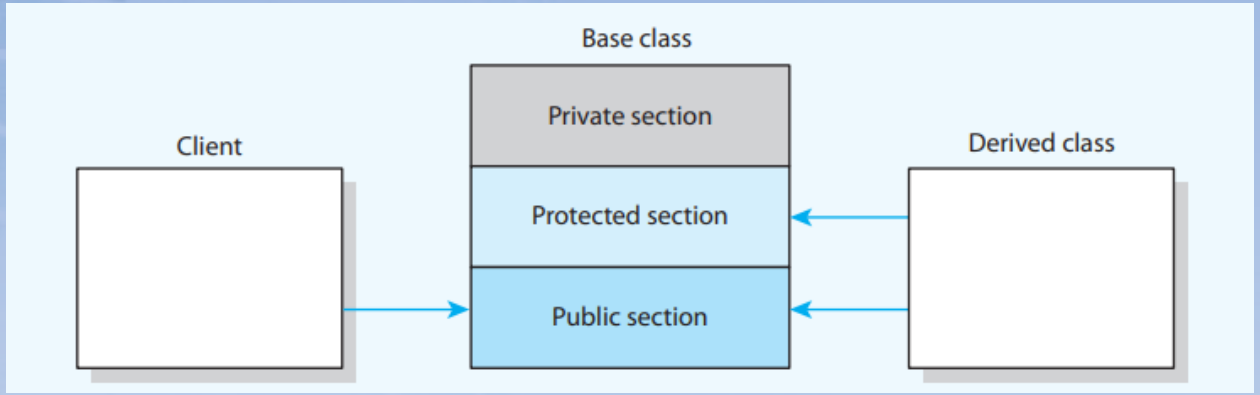
© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Exceptions - Summary

- To throw an exception in a function:
  - Determine what errors, preconditions, postconditions, and unusual conditions you must check and where they should be checked, so your function can correctly perform its task. Usually this is done with an if statement.
  - If an error condition has occurred, throw an exception using the throw statement. As an argument to the exception class constructor, pass a string description of the reason for the error.
  - Add a throw clause to the function header indicating the exceptions thrown by the function.
  - Derive a custom exception class to better identify the conditions that caused the error, though doing so is optional, it is desirable.
- To handle an exception or use a function that could throw an exception
  - Place the statement that might throw an exception in a try block. The fewer statements in the try block the better.
  - Do not use new to create an object in a try block.
  - After the try block, place catch blocks for each type of exception that can be thrown by the statements in the try block.
  - Place the catch blocks in order from the most specific exception classes to the more general exception classes.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Public, Private and Protected



- Public members can be used by anyone.
- Private members can be used only by methods of the class.
- Protected members can be used only by methods of the class or any derived class.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

End